

StackableGui — Complete Documentation

Version: 1.0

Namespace: Sparkling.StackableGui

Unity: 2022.3 LTS or newer recommended

Table of Contents

1. [Philosophy & Design Goals](#)
2. [Core Concepts](#)
3. [Architecture Overview](#)
4. [Canvas Layers](#)
5. [Canvas Settings](#)
6. [Asset Loading](#)
7. [Implementing a UI Element](#)
8. [The Director](#)
9. [Push & Pop](#)
10. [Insert & Remove](#)
11. [Visibility Modes](#)
12. [Input Blocking](#)
13. [Querying the Stack](#)
14. [Stack Events](#)
15. [Safe Area](#)
16. [Screen Shake](#)
17. [Resolution Hot-Swap](#)
18. [Integrating with a MonoBehaviour Director](#)
19. [Common Patterns & Recipes](#)
20. [Pitfalls & Gotchas](#)

1. Philosophy & Design Goals

StackableGui was born from a single observation: **UI management in Unity games tends to become a tangled web of `SetActive` calls, hard-coded references, and invisible dependencies between screens.** As a project grows, every new popup, every transition, every "show this only when that other panel is closed" rule adds another knot to that web.

The solution is not a smarter way to call `SetActive`. The solution is to **stop thinking about UI panels as objects you show and hide, and start thinking about them as items you push onto and pop off a stack** — the same mental model that has driven screen navigation in mobile operating systems for decades.

StackableGui is built on four principles:

1.1 Stack Semantics Over Booleans

Every canvas layer in StackableGui is an independent stack. You do not ask "is the pause menu visible?". You ask "is there anything on the System canvas?". You do not toggle a pause menu on and off — you push it on, and pop it off. This makes the state of your entire UI unambiguous and inspectable at any point.

1.2 The Director Knows Everything, Elements Know Themselves

The `IStableableUiDirector` is the single point of truth for the UI state. No element needs a reference to any other element. No panel needs to know what is behind it. Elements only need to know how to show and hide themselves — the director handles ordering, sorting, and blocking.

This creates a clean boundary:

- **Director** → *where things are, in what order, which receives input*
- **Element** → *what it looks like, how it animates, what it does*

1.3 Composition Over Configuration

Rather than a monolithic UI manager that tries to anticipate every possible need through a sea of booleans and Inspector fields, StackableGui provides small, orthogonal building blocks: a visibility mode, a blocking mode, a canvas type. You compose them per-call, giving you precise control without a bloated API surface.

1.4 No Framework Lock-In

StackableGui does not care how you load assets. It does not care how you animate. It does not care whether you use Addressables, `Resources`, a custom pool, or a raw `Instantiate`. The `IUiAssetLoader` interface is the only seam between the system and your project's infrastructure — swap it out at any time without touching the rest of the codebase.

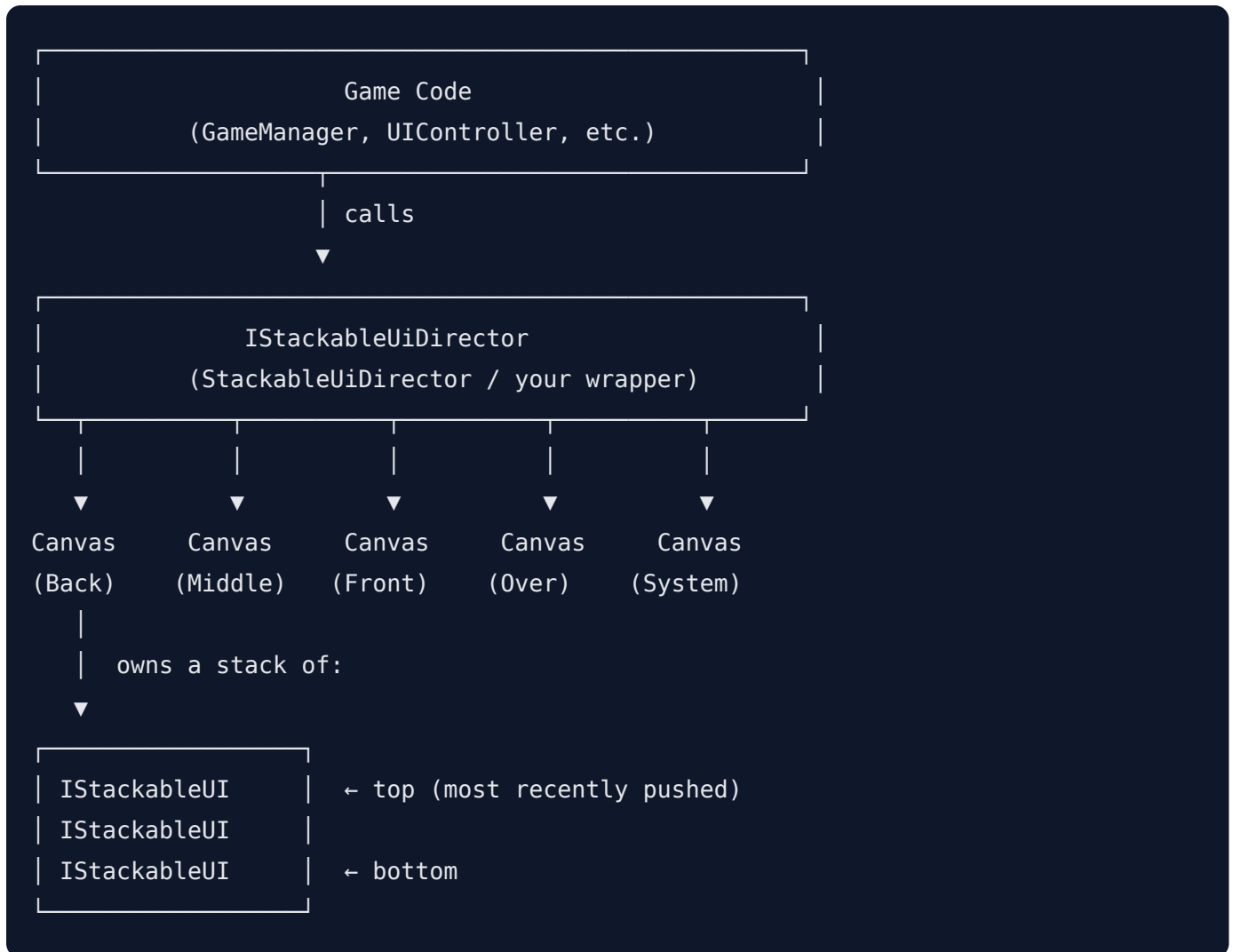
2. Core Concepts

Before diving into code, here is a glossary of the key terms used throughout this documentation.

Term	Meaning
Element	Any piece of UI managed by this system. Implements <code>IStableableUIElement</code> . In practice it is a C# class (not a <code>MonoBehaviour</code>) that wraps a prefab.

Canvas	A Unity <code>Canvas</code> component plus its associated <code>StackableUiCanvas</code> <code>MonoBehaviour</code> . Owns one independent stack of elements.
Layer	Synonym for a canvas in the context of the <code>CanvasType</code> hierarchy.
Director	The <code>StackableUiDirector</code> (or your <code>MonoBehaviour</code> wrapper). The single API you call from game code.
Push	Load an element's prefab, instantiate it, add it to the top of a canvas stack.
Pop	Remove the top element from a canvas stack and notify it so it can animate out and destroy itself.
Stack	The ordered collection of elements in a single canvas layer. The top of the stack is the element most recently pushed.
Visibility Mode	A per-operation flag that controls whether only the top element or all elements in the stack are active.
Blocking Mode	A per-operation flag that controls which canvases receive pointer input after the operation.

3. Architecture Overview



The director holds a `Dictionary<CanvasType, StackableUiCanvas>`. Every operation goes through the director, which looks up the target canvas and delegates to it. The canvas owns the stack as a `LinkedList<IStackableUIElement>` — chosen deliberately for $O(1)$ add/remove at both ends and $O(n)$ traversal for sorting.

Critically, **the director is a plain C# class**, not a `MonoBehaviour`. This means it has no ties to the Unity lifecycle and can be constructed, tested, and swapped freely. The `MonoBehaviour` wrapper (`SampleStackableGuiDirector` in the sample) is just a thin host that provides the `Transform` parent and holds the director as a field.

4. Canvas Layers (`CanvasType`)

Every canvas in `StackableGui` maps to a value in the `CanvasType` enum. The integer value of each entry directly determines its sorting order relative to the others.

```
public enum CanvasType
{
    Background = 1,
    Back       = 2,
    Middle     = 3,
    Front      = 4,
    Over       = 5,
    System     = 6,
    Loading    = 7
}
```

The actual Unity sorting order applied to each canvas is:

```
sortingOrder = BaseOrder + (OrderStep × (int)CanvasType)
```

With `CanvasSetting.Default (BaseOrder = 0, OrderStep = 10)`:

Layer	Sorting Order	Intended Use
Background	10	Environment backgrounds, parallax panoramas
Back	20	Secondary panels sitting behind main UI
Middle	30	Primary HUD and gameplay UI
Front	40	Popups, modals, confirmation dialogs
Over	50	Tooltips, floating labels, temporary notifications
System	60	Pause menus, settings screens, global overlays
Loading	70	Loading and transition screens — always on top

Choosing the Right Layer

The rule of thumb is simple: **ask "should this element appear above everything currently on screen?"** If yes, go one layer higher than the highest occupied layer. If the element belongs to a well-defined category (popup, tooltip, loading), use the matching layer directly.

```
// A HUD element: lives in the primary gameplay layer.
director.PushUiElementIntoCanvas(myHud, CanvasType.Middle);

// A popup triggered by the HUD: must appear above it.
director.PushUiElementIntoCanvas(myPopup, CanvasType.Front);

// A tooltip triggered by something in the popup:
// must appear above the popup too.
director.PushUiElementIntoCanvas(myTooltip, CanvasType.Over);

// The pause menu must cover everything – even tooltips.
director.PushUiElementIntoCanvas(pauseMenu, CanvasType.System);
```

The `OrderStep` gap (default: 10) between layers means you can have sub-sorting within a layer using Unity's own `Canvas.sortingOrder` override on child canvases, without ever conflicting with the layer boundaries.

5. Canvas Settings (`CanvasSetting`)

`CanvasSetting` is a serializable struct that fully describes how the root canvas and its scaler should be configured. It is passed to the director at construction time and can be changed at runtime via `ChangeCanvasResolution`.

Built-in Presets

```
// PC / console: 1920×1080, balanced width-height match.
CanvasSetting.Default

// Portrait mobile: 1080×1920, match width only (safe for narrow screens).
CanvasSetting.Mobile

// High-DPI monitors: 2560×1440, 200 pixels per unit.
CanvasSetting.HighDpi
```

Custom Settings

```

var settings = new CanvasSetting
{
    CanvasRenderMode      = RenderMode.ScreenSpaceOverlay,
    ScaleMode              = CanvasScaler.ScaleMode.ScaleWithScreenSize,
    MatchMode              = CanvasScaler.ScreenMatchMode.MatchWidthOrHeight,
    ReferenceResolution    = new Vector2(1920f, 1080f),
    Match                  = 0.5f,          // 0 = match width, 1 = match height
    ReferencePixelsPerUnit = 100f,
    BaseOrder              = 0,
    OrderStep              = 10             // must be > 0
};

var director = new StackableUiDirector(settings, new MyLoader());

```

The OrderStep Field

`OrderStep` must be strictly positive. It is the gap in sorting order between consecutive layers. A value of `10` (default) is a safe choice: it leaves room for custom sub-sorting within a layer while ensuring layers never overlap each other in depth.

If you use `overrideSorting` on individual elements within a canvas, ensure the override values stay within the `[0, OrderStep)` range to avoid bleeding into the next layer.

6. Asset Loading (`IUiAssetLoader`)

The `IUiAssetLoader` interface is the only point of contact between `StackableGui` and your project's asset pipeline. Implementing it is mandatory — and intentionally kept minimal.

```

public interface IUiAssetLoader
{
    GameObject LoadAsset(string path);
    void LoadAssetAsync(string path, Action<GameObject> onLoaded,
        Action<Exception> onError = null);
    void ReleasePrefab(GameObject prefab);
}

```

6.1 Resources Loader (reference implementation)

The sample project ships with `SampleResourcesLoader`, which wraps Unity's built-in `Resources` API. Use it as a starting point:

```

public class ResourcesLoader : IUiAssetLoader
{
    public GameObject LoadAsset(string path)
    {
        var prefab = Resources.Load<GameObject>(path);
        if (prefab == null)
            Debug.LogError($"[ResourcesLoader] Prefab not found at: '{path}'");
        return prefab;
    }

    public void LoadAssetAsync(string path, Action<GameObject> onLoaded,
        Action<Exception> onError = null)
    {
        var op = Resources.LoadAsync<GameObject>(path);
        op.completed += _ =>
        {
            if (op.asset is GameObject prefab)
                onLoaded?.Invoke(prefab);
            else
                onError?.Invoke(new Exception($"Prefab not found: '{path}'"));
        };
    }

    public void ReleasePrefab(GameObject prefab)
    {
        // Resources prefabs don't need explicit release.
        // With Addressables, call Addressables.Release(prefab) here.
    }
}

```

6.2 Addressables Loader


```

using UnityEngine.AddressableAssets;
using UnityEngine.ResourceManagement.AsyncOperations;

public class AddressablesLoader : IUiAssetLoader
{
    // Track handles so we can release them properly.
    private readonly Dictionary<GameObject, AsyncOperationHandle<GameObject>>
m_handles
        = new Dictionary<GameObject, AsyncOperationHandle<GameObject>>();

    public GameObject LoadAsset(string path)
    {
        // Synchronous load is discouraged with Addressables.
        // Prefer the async variant in all production paths.
        var handle = Addressables.LoadAssetAsync<GameObject>(path);
        handle.WaitForCompletion();

        if (handle.Status != AsyncOperationStatus.Succeeded)
        {
            Debug.LogError($"[AddressablesLoader] Failed to load '{path}'");
            return null;
        }

        m_handles[handle.Result] = handle;
        return handle.Result;
    }

    public void LoadAssetAsync(string path, Action<GameObject> onLoaded,
Action<Exception> onError = null)
    {
        var handle = Addressables.LoadAssetAsync<GameObject>(path);
        handle.Completed += h =>
        {
            if (h.Status == AsyncOperationStatus.Succeeded)
            {
                m_handles[h.Result] = h;
                onLoaded?.Invoke(h.Result);
            }
            else
            {
                onError?.Invoke(new Exception($"Addressables failed for '{path}':
{h.OperationException}"));
            }
        }
    }
}

```

```

        }
    };
}

public void ReleasePrefab(GameObject prefab)
{
    if (m_handles.TryGetValue(prefab, out var handle))
    {
        Addressables.Release(handle);
        m_handles.Remove(prefab);
    }
}
}

```

6.3 Object Pool Loader

If you already have an object pool and want StackableGui to reuse instances rather than instantiating fresh ones, implement both `LoadAsset` (check-out from pool) and `ReleasePrefab` (return to pool):

```

public class PooledLoader : IUiAssetLoader
{
    private readonly IObjPool<string, GameObject> m_pool;

    public PooledLoader(IObjPool<string, GameObject> pool)
        => m_pool = pool;

    public GameObject LoadAsset(string path)           => m_pool.Get(path);
    public void ReleasePrefab(GameObject prefab)        => m_pool.Return(prefab);

    public void LoadAssetAsync(string path, Action<GameObject> onLoaded,
        Action<Exception> onError = null)
    {
        // If your pool supports async warm-up, use it here.
        // Otherwise, fall back to synchronous get.
        try { onLoaded?.Invoke(m_pool.Get(path)); }
        catch (Exception e) { onError?.Invoke(e); }
    }
}

```

7. Implementing a UI Element (`IStackableUIElement`)

Every piece of UI managed by StackableGui must implement `IStackableUIElement`. This is the contract between the director and your screen controllers. The interface is intentionally lean — it describes *existence* and *lifecycle*, not specific UI behaviour.

```
public interface IStackableUIElement
{
    string      Path      { get; }    // passed verbatim to IUiAssetLoader
    bool        IsVisible { get; }
    bool        IsLoaded  { get; }
    bool        IsAnimating { get; }
    GameObject Instance { get; }    // the live GameObject in the scene
    GameObject Prefab   { get; }    // the source prefab (used for release)
    GameObject Parent   { get; }    // the canvas GameObject

    void Initialize(GameObject prefab, GameObject parent); // called by director
on push
    void OnPushedIntoStack();                                // called after
Initialize
    void OnPoppedFromStack();                                // called before
release
    void SetActive(bool active);                             // called by
visibility logic
    void Animate(string animationName, Action callback = null);
}
```

7.1 Minimal Implementation

```

public class MyPanel : IStackableUIElement
{
    public string      Path          => "UI/MyPanel";
    public bool        IsVisible     => Instance != null && Instance.activeSelf;
    public bool        IsLoaded      => Instance != null;
    public bool        IsAnimating   => false;
    public GameObject Instance       { get; private set; }
    public GameObject Prefab         { get; private set; }
    public GameObject Parent         { get; private set; }

    public void Initialize(GameObject prefab, GameObject parent)
    {
        Prefab    = prefab;
        Parent    = parent;
        Instance = Object.Instantiate(prefab, parent.transform);
    }

    public void OnPushedIntoStack()
    {
        // Subscribe to button events, start an enter animation, etc.
    }

    public void OnPoppedFromStack()
    {
        // Unsubscribe, play exit animation, then destroy.
        Object.Destroy(Instance);
    }

    public void SetActive(bool active) => Instance.SetActive(active);
    public void Animate(string name, Action callback = null) { callback?.Invoke(); }
}

```

7.2 Using the Base Class

The project ships with `SampleStackableElement`, a ready-made abstract base class that handles `Initialize`, `SetActive`, `Animate` (via `Animator`), and `Destroy`. Extend it for a faster start:

```

public class MainMenuElement : SampleStackableElement
{
    public override string Path      => "UI/MainMenu";
    public override bool   IsLoaded => m_renderer != null && !m_renderer.cull;

    private CanvasRenderer m_renderer;
    private Button         m_playButton;

    public override void Initialize(GameObject prefab, GameObject parent)
    {
        base.Initialize(prefab, parent); // handles Instantiate + Animator

        m_renderer = m_instance.GetComponentInChildren<CanvasRenderer>();
        m_playButton =
m_instance.transform.Find("PlayButton").GetComponent<Button>();
    }

    public override void OnPushedIntoStack()
    {
        m_playButton.onClick.AddListener(OnPlayClicked);
        Animate("Enter"); // drives the Animator on the prefab
    }

    public override void OnPoppedFromStack()
    {
        m_playButton.onClick.RemoveAllListeners();
        Animate("Exit", Destroy); // Destroy is provided by SampleStackableElement
    }

    private void OnPlayClicked()
    {
        // Notify game code via event or callback injected at construction.
    }
}

```

7.3 Injecting Dependencies

Elements are plain C# objects, which means you can inject any dependency at construction time — a pattern far cleaner than finding components after the fact:

```

public class PauseMenuElement : SampleStackableElement
{
    public override string Path => "UI/PauseMenu";

    private readonly Action m_onResume;
    private readonly Action m_onQuit;

    // Dependencies injected via constructor – no singletons needed.
    public PauseMenuElement(Action onResume, Action onQuit)
    {
        m_onResume = onResume;
        m_onQuit    = onQuit;
    }

    public override void OnPushedIntoStack()
    {
        var resumeBtn =
m_instance.transform.Find("ResumeButton").GetComponent<Button>();
        var quitBtn   =
m_instance.transform.Find("QuitButton").GetComponent<Button>();

        resumeBtn.onClick.AddListener(() => m_onResume?.Invoke());
        quitBtn.onClick.AddListener(() => m_onQuit?.Invoke());

        Animate("Enter");
    }

    public override void OnPoppedFromStack()
    {
        Animate("Exit", Destroy);
    }
}

```

8. The Director (IStackableUiDirector)

The director is the single object your game code interacts with. It exposes every operation the system supports and enforces all invariants (initialization check, canvas existence check, duplicate element check).

8.1 Construction and Initialization

The director is a plain C# class. Construction does not touch Unity APIs — the canvas hierarchy is built separately via `GenerateCanvases` .

```
// Construction is safe outside of Unity's lifecycle.
var director = new StackableUiDirector(CanvasSetting.Default, new MyLoader());

// GenerateCanvases must be called on the main thread (it creates GameObjects).
// Pass the Transform that will parent all the canvas GameObjects.
director.GenerateCanvases(myParentTransform);
```

After `GenerateCanvases` returns, `director.Initialized` is `true` and every `CanvasType` has a live canvas ready to receive elements.

8.2 Checking Initialization

All director methods silently guard against being called before initialization:

```
if (!director.Initialized)
{
    // Methods called here will log a warning and return early.
    // Nothing will crash – but nothing will happen either.
}
```

In production code, initialization happens in `Start` or `Awake` of a `MonoBehaviour` host, so by the time any game system calls the director it is always ready. The guard exists for tooling and unit-test scenarios.

9. Push & Pop

Push and Pop are the bread and butter of the system. Every operation comes in two flavours: **synchronous** (loads the prefab via `LoadAsset`) and **async-with-callback** (loads via `LoadAssetAsync` , then invokes a callback once the element is live).

9.1 Synchronous Push

Use when your loader's `LoadAsset` is guaranteed to return immediately (e.g. `Resources.Load` with prewarmed cache, or a pool that always has the asset ready).

```
var element = new MainMenuElement();
director.PushUiElementIntoCanvas(element, CanvasType.Middle);
// element.OnPushedIntoStack() is called before this line returns.
```

9.2 Async Push with Callback

Use for Addressables, async pools, or any loader that might take more than one frame. The callback fires after `OnPushedIntoStack` is called on the element.

```
var popup = new ConfirmationPopup();

director.PushUiElementIntoCanvasCallback(
    element:      popup,
    type:         CanvasType.Front,
    mode:         StackVisibilityMode.AllVisible,
    blockingMode: InputBlockingMode.BlockBelowTop,
    callback:     () =>
    {
        // The prefab is instantiated, the element is in the stack,
        // and OnPushedIntoStack has already been called.
        // This is the right place to start animations or subscribe further.
        popup.Animate("Enter");
    }
);
```

9.3 Pop

```
// Pop the top element unconditionally.
director.PopUiElementFromCanvas(CanvasType.Front);

// Pop only if the top element is of the given type.
// Safe no-op if the top element is something else.
director.PopUiElementFromCanvasIfMatch<ConfirmationPopup>(CanvasType.Front);
```

9.4 Pop with Callback


```

director.PopUiElementFromCanvasCallback(
  type:      CanvasType.Front,
  callback: () =>
  {
    // Fires after OnPoppedFromStack – but note: if your OnPoppedFromStack
    // plays an animation and destroys asynchronously, this callback fires
    // before the animation finishes. Coordinate via your element's own
    // callback if you need to wait for the exit animation.
    Debug.Log("Popup removed from stack.");
  }
);

// Type-safe variant:
director.PopUiElementFromCanvasCallbackIfMatch<ConfirmationPopup>(
  type:      CanvasType.Front,
  callback: () => Debug.Log("Confirmation popup removed.")
);

```

9.5 Full Push-Pop Round Trip

A common pattern for a modal dialog that must animate in, wait for user input, then animate out:

```
// 1. Create the element with its callbacks injected.
var dialog = new ConfirmationPopup(
    onConfirm: () =>
    {
        // User confirmed – pop the dialog.
        director.PopUiElementFromCanvasIfMatch<ConfirmationPopup>
(CanvasType.Front);
        // Then do whatever the confirmation triggers.
        StartGame();
    },
    onCancel: () =>
    {
        director.PopUiElementFromCanvasIfMatch<ConfirmationPopup>
(CanvasType.Front);
    }
);

// 2. Push it. The callback fires once the prefab is live.
director.PushUiElementIntoCanvasCallback(
    element:      dialog,
    type:         CanvasType.Front,
    mode:         StackVisibilityMode.AllVisible,
    blockingMode: InputBlockingMode.BlockBelowTop,
    callback:     () => dialog.Animate("Enter")
);
```

10. Insert & Remove

Push and Pop always operate on the top of the stack. Insert and Remove give you surgical access to any position.

10.1 Insert at Index

```
// Index 0 = bottom of the stack, Count = top.
director.InsertUiElementInCanvas(
    element: myBackground,
    index:   0,                // insert below everything else
    type:    CanvasType.Middle
);
```

Insert is synchronous: it calls `LoadAsset` and throws `IndexOutOfRangeException` if the index is out of bounds. If you need async insertion, use `StackableUiCanvas.InsertUiElementCallback` directly (exposed for advanced use).

When to use Insert over Push:

Insert is for cases where the stack order matters independently of arrival time. For example, if you push a HUD and then decide a background panel should live beneath it without clearing and rebuilding the stack:

```
// HUD is already at index 0 (top, since it is the only element).
// Insert the background below it.
director.InsertUiElementInCanvas(background, index: 0, CanvasType.Middle);
// Stack is now: [background(0), hud(1)]
```

10.2 Remove a Specific Element

```
// Remove an element regardless of where in the stack it sits.
director.RemoveUiElementFromCanvas(myBackground, CanvasType.Middle);
```

`RemoveUiElementFromCanvas` calls `OnPoppedFromStack` on the element and releases its prefab, exactly like `Pop` — the only difference is that it finds the element by reference rather than taking from the top.

11. Visibility Modes (`StackVisibilityMode`)

Every `Push`, `Pop`, `Insert`, and `Remove` accepts a `StackVisibilityMode` parameter. It is applied to the entire stack after the operation completes.

```
public enum StackVisibilityMode
{
    TopOnly,    // Only the top element is SetActive(true). All others:
                SetActive(false).
    AllVisible  // Every element in the stack is SetActive(true).
}
```

`AllVisible` — Default

All elements stay active. Use this when stacked elements are intentionally visible at the same time: a HUD below a semi-transparent popup, a background image below a menu.

```
// Both the menu and the background image behind it stay visible.
director.PushUiElementIntoCanvas(
    menu, CanvasType.Middle, StackVisibilityMode.AllVisible
);
```

TopOnly — Exclusive Focus

The moment a new element is pushed, all elements below it are deactivated. When the top element is popped, the new top is reactivated. This is the classic mobile navigation pattern where only one screen is ever visible.

```
director.PushUiElementIntoCanvas(
    settingsScreen, CanvasType.System, StackVisibilityMode.TopOnly
);
// All previously active elements in System canvas are now inactive.

director.PopUiElementFromCanvas(CanvasType.System, StackVisibilityMode.TopOnly);
// The element below settingsScreen is now reactivated.
```

Important: `StackVisibilityMode` is applied *at the time of the operation*. The system does not remember which mode was used to push an element — it re-applies the mode you pass to each `Pop`, too. Always pass the same mode consistently for a given canvas layer, or manage visibility manually between calls.

12. Input Blocking (`InputBlockingMode`)

The input blocking mode controls the `GraphicRaycaster` on each canvas after an operation. This determines which canvas layers can receive pointer events.

```
public enum InputBlockingMode
{
    BlockNone,        // All canvases receive input.
    BlockBelowTop,    // Only the highest canvas that has elements receives input.
    BlockAll           // No canvas receives input.
}
```

BlockNone — Default

Every canvas with a `GraphicRaycaster` processes input. Use this when multiple layers legitimately need to be interactive at once.

BlockBelowTop — Modal Behaviour

Only the topmost occupied canvas receives input. Any canvas below it has its `GraphicRaycaster` disabled. This is the correct mode for popups and modals that must prevent accidental interaction with the UI behind them.

```
// Push a popup and block all input to canvases below it.
director.PushUiElementIntoCanvasCallback(
    element:      confirmationPopup,
    type:         CanvasType.Front,
    mode:         StackVisibilityMode.AllVisible,
    blockingMode: InputBlockingMode.BlockBelowTop,
    callback:     () => confirmationPopup.Animate("Enter")
);

// When popped, restore full input.
director.PopUiElementFromCanvasIfMatch<ConfirmationPopup>(
    type:         CanvasType.Front,
    blockingMode: InputBlockingMode.BlockNone
);
```

BlockAll — Cinematic / Loading

Disable all input across every canvas. Use during loading screens or cutscenes where no interaction should be possible.

```
director.PushUiElementIntoCanvas(
    loadingScreen, CanvasType.Loading,
    blockingMode: InputBlockingMode.BlockAll
);
```

13. Querying the Stack

Check if a Canvas Has Elements

```
if (director.AnyElementInCanvas(CanvasType.Front))
{
    Debug.Log("There is at least one popup open.");
}
```

Find the First Element of a Type

```
if (director.TryFindElementInCanvas<ConfirmationPopup>(out var popup,
CanvasType.Front))
{
    popup.SetTitle("Are you sure?");
}
```

Find All Elements of a Type

```
if (director.TryFindElementsInCanvas<TooltipElement>(out var tooltips,
CanvasType.Over))
{
    foreach (var tooltip in tooltips)
    {
        tooltip.Hide();
    }
}
```

Check if an Element is Animating

`IsAnimating` is exposed on `IStackableUIElement` and tracked by `SampleStackableElement`. Use it to avoid overlapping pushes/pops while a transition is in flight:

```
if (director.TryFindElementInCanvas<MainMenuElement>(out var menu,
CanvasType.Middle))
{
    if (!menu.IsAnimating)
    {
        // Safe to push another element on top.
        director.PushUiElementIntoCanvas(settingsScreen, CanvasType.System);
    }
}
```

Clear Operations

```
// Remove all elements from a specific canvas.  
// OnPoppedFromStack is called on every element.  
director.ClearCanvas(CanvasType.Front);  
  
// Remove all elements from every canvas simultaneously.  
director.ClearAllCanvases();  
  
// Destroy the canvas GameObject entirely and remove it from the director.  
// Useful for unloading a scene's UI layer without affecting others.  
director.DestroyCanvas(CanvasType.Loading);
```

14. Stack Events (OnStackChanged)

`StackableUiCanvas` fires `OnStackChanged` after every mutation to its stack. The event delivers a `StackChangedEventArgs` struct:

```
public readonly struct StackChangedEventArgs  
{  
    public readonly StackChangeType    ChangeType;    // Pushed, Popped, or  
    Cleared  
    public readonly IStackableUIElement Element;      // null when ChangeType is  
    Cleared  
    public readonly int                OldStackSize;  
    public readonly int                NewStackSize;  
}
```

Subscribing

To subscribe, you need a reference to the `StackableUiCanvas` — which is not exposed directly through `IStackableUiDirector` by default (to keep the interface lean). Wire it up in your `MonoBehaviour` director wrapper:

```
// In your MonoBehaviour director wrapper, after GenerateCanvases:
public void SubscribeToCanvas(CanvasType type, Action<StackChangedEventArgs>
handler)
{
    if (m_director.TryGetCanvas(type, out var canvas))
    {
        canvas.OnStackChanged += handler;
    }
}
```

Or expose it directly on your wrapper:

```
public event Action<StackChangedEventArgs> OnFrontCanvasChanged
{
    add    => m_frontCanvas.OnStackChanged += value;
    remove => m_frontCanvas.OnStackChanged -= value;
}
```

Use Cases


```
// Log every stack mutation for debugging.
canvas.OnStackChanged += args =>
{
    Debug.Log($"[{args.ChangeType}] on {canvasType} - " +
        $"stack {args.OldStackSize} → {args.NewStackSize}" +
        (args.Element != null ? $" ({args.Element.GetType().Name})" : ""));
};

// Update a popup counter badge on the HUD.
frontCanvas.OnStackChanged += args =>
{
    hudElement.SetPopupCount(args.NewStackSize);
};

// Analytics event on every screen push.
canvas.OnStackChanged += args =>
{
    if (args.ChangeType == StackChangeType.Pushed)
    {
        Analytics.TrackScreen(args.Element.GetType().Name);
    }
};
```

15. Safe Area

Safe Area support ensures that UI elements respect the insets imposed by device notches, punch holes, rounded corners, and home indicators — especially critical on iOS and modern Android devices.

How It Works

After `GenerateCanvases`, each sub-canvas has its `RectTransform` anchors set to match `Screen.safeArea` proportions. For `ScreenSpaceOverlay` this is simple arithmetic; for `ScreenSpaceCamera` the safe area corners are converted through the camera's viewport.

Initial Application

Safe area is applied automatically during `GenerateCanvases`. If the device orientation is fixed at startup, you never need to call it again.

```
director.GenerateCanvases(transform);
// Safe area already applied to all canvases.
```

Handling Orientation Changes

If your game supports rotation, call `ApplySafeArea` whenever the orientation changes. The method is idempotent — it compares the current `Screen.safeArea` to the cached value and skips the work if nothing has changed.

```
// In your MonoBehaviour director wrapper:
private void Update()
{
    // Poll for safe area changes (orientation, foldable display changes).
    m_director.ApplySafeArea();
}
```

Or, more efficiently, react to Unity's screen orientation event:

```
private ScreenOrientation m_lastOrientation;

private void Update()
{
    if (Screen.orientation != m_lastOrientation)
    {
        m_lastOrientation = Screen.orientation;
        m_director.ApplySafeArea();
    }
}
```

Checking if Safe Area Is Active

```
if (director.HasSafeArea)
{
    // The device has a notch or cutout affecting the UI layout.
    // You might want to adjust custom UI elements that are anchored
    // to screen edges.
}
```

16. Screen Shake

StackableGui includes a Perlin-noise screen shake implementation that operates on the canvas's `RectTransform` position. Shaking a canvas rather than the camera avoids interfering with the game world and is safe to combine with camera-based effects.

```
/// <param name="duration"> Total shake duration in seconds.      </param>
/// <param name="magnitude"> Peak displacement in canvas units.  </param>
/// <param name="frequency"> Oscillation speed (higher = faster). </param>
director.ShakeCanvas(CanvasType.Middle, duration: 0.5f, magnitude: 8f, frequency:
25f);
director.ShakeAllCanvases(                                     duration: 1.0f, magnitude:
5f, frequency: 30f);
```

Notes

- Shake uses `Time.unscaledDeltaTime`, so it works correctly when `Time.timeScale` is 0 (e.g. during pause menus).
- Displacement is damped linearly over the duration — the shake starts at full magnitude and tapers to zero.
- If `ShakeCanvas` is called while a shake is already in progress on that canvas, the new call is silently ignored. The in-flight shake completes first.
- After the shake completes, the canvas position is restored to its original value exactly.

Practical Examples

```
// Light UI feedback when the player clicks a locked button.
director.ShakeCanvas(CanvasType.Middle, duration: 0.25f, magnitude: 4f, frequency:
40f);

// Heavy impact when the player takes a big hit.
director.ShakeAllCanvases(duration: 0.6f, magnitude: 12f, frequency: 20f);

// Subtle notification pulse.
director.ShakeCanvas(CanvasType.Over, duration: 0.15f, magnitude: 2f, frequency:
60f);
```

17. Resolution Hot-Swap

You can change the canvas scaler settings at runtime without rebuilding the canvas hierarchy. This is useful for supporting multiple resolutions or orientation switches in games that change layout significantly between portrait and landscape.

```
// Switch to mobile portrait layout at runtime.
director.ChangeCanvasResolution(CanvasSetting.Mobile);

// Switch back to desktop layout.
director.ChangeCanvasResolution(CanvasSetting.Default);

// Apply a fully custom setting.
director.ChangeCanvasResolution(new CanvasSetting
{
    CanvasRenderMode      = RenderMode.ScreenSpaceOverlay,
    ScaleMode              = CanvasScaler.ScaleMode.ScaleWithScreenSize,
    ReferenceResolution    = new Vector2(2732f, 2048f), // iPad Pro
    Match                  = 0.5f,
    ReferencePixelsPerUnit = 100f,
    BaseOrder              = 0,
    OrderStep              = 10
});
```

`ChangeCanvasResolution` only modifies the `CanvasScaler` on the root (parent) canvas — it does not touch sub-canvases or any element instances.

18. Integrating with a MonoBehaviour Director

`StackableUiDirector` is a plain C# class. To make it accessible across your project, wrap it in a `MonoBehaviour` singleton that is placed in the scene. The sample project provides `SampleStackableGuiDirector` as a complete reference.

Minimal MonoBehaviour Wrapper

```

using System;
using UnityEngine;
using Sparkling.StackableGui;

public class GuiDirector : MonoBehaviour, IStackableUiDirector
{
    public static GuiDirector Instance { get; private set; }

    [SerializeField] private CanvasSetting m_settings = CanvasSetting.Default;

    private StackableUiDirector m_director;

    public bool Initialized => m_director.Initialized;
    public bool HasSafeArea => m_director.HasSafeArea;

    private void Awake()
    {
        Instance = this;
        m_director = new StackableUiDirector(m_settings, new ResourcesLoader());
    }

    private void Start()
    {
        m_director.GenerateCanvases(transform);
    }

    private void OnDestroy()
    {
        foreach (CanvasType type in Enum.GetValues(typeof(CanvasType)))
            m_director.DestroyCanvas(type);
    }

    // Delegate every IStackableUiDirector member to m_director.
    public void PushUiElementIntoCanvas(IStackableUIElement e, CanvasType t,
        StackVisibilityMode m = StackVisibilityMode.AllVisible,
        InputBlockingMode b = InputBlockingMode.BlockNone)
        => m_director.PushUiElementIntoCanvas(e, t, m, b);

    // ... repeat for all other interface members.
}

```

Accessing the Director from Game Code

```
// From anywhere in your project:  
GuiDirector.Instance.PushUiElementIntoCanvas(new MainMenuElement(),  
CanvasType.Middle);
```

Or, for better testability, inject the director as a dependency rather than using the singleton directly:

```
public class GameStateManager  
{  
    private readonly IStackableUiDirector m_ui;  
  
    public GameStateManager(IStackableUiDirector ui)  
    {  
        m_ui = ui;  
    }  
  
    public void OpenPauseMenu()  
    {  
        m_ui.PushUiElementIntoCanvasCallback(  
            new PauseMenuElement(Resume, Quit),  
            CanvasType.System,  
            StackVisibilityMode.AllVisible,  
            InputBlockingMode.BlockBelowTop,  
            callback: null  
        );  
    }  
  
    private void Resume() => m_ui.PopUiElementFromCanvasIfMatch<PauseMenuElement>  
(CanvasType.System);  
    private void Quit()    => Application.Quit();  
}
```

19. Common Patterns & Recipes

Pattern 1: Main Menu Flow with Back Navigation

```

public class MainMenuFlow
{
    private readonly IStackableUiDirector m_ui;

    public MainMenuFlow(IStackableUiDirector ui) => m_ui = ui;

    public void Start()
    {
        var mainMenu = new MainMenuElement(
            onSettings: OpenSettings,
            onPlay:      OpenCharacterSelect
        );
        m_ui.PushUiElementIntoCanvasCallback(
            mainMenu, CanvasType.Middle,
            StackVisibilityMode.TopOnly, InputBlockingMode.BlockNone,
            () => mainMenu.Animate("Enter")
        );
    }

    private void OpenSettings()
    {
        var settings = new SettingsElement(onBack: GoBack);
        m_ui.PushUiElementIntoCanvasCallback(
            settings, CanvasType.Middle,
            StackVisibilityMode.TopOnly, InputBlockingMode.BlockNone,
            () => settings.Animate("SlideIn")
        );
    }

    private void OpenCharacterSelect()
    {
        var charSelect = new CharacterSelectElement(onBack: GoBack, onConfirm:
StartGame);
        m_ui.PushUiElementIntoCanvasCallback(
            charSelect, CanvasType.Middle,
            StackVisibilityMode.TopOnly, InputBlockingMode.BlockNone,
            () => charSelect.Animate("SlideIn")
        );
    }

    // Universal back handler: just pop the top of the Middle canvas.
    private void GoBack()

```

```
=> m_ui.PopUiElementFromCanvas(CanvasType.Middle,  
StackVisibilityMode.TopOnly);  
  
private void StartGame() { /* load game scene */ }  
}
```

Pattern 2: Layered HUD with Semi-Transparent Popup

```
// HUD and map panel both visible at the same time.  
var hud = new HudElement();  
m_ui.PushUiElementIntoCanvas(hud, CanvasType.Middle,  
StackVisibilityMode.AllVisible);  
  
var mapPanel = new MapPanelElement();  
m_ui.PushUiElementIntoCanvas(mapPanel, CanvasType.Middle,  
StackVisibilityMode.AllVisible);  
// Both HUD and map are active – map is on top but HUD is still visible behind it.
```

Pattern 3: Confirmation Dialog Gate


```

private void OnQuitButtonClicked()
{
    // Block input to everything behind the dialog.
    var dialog = new ConfirmationDialog(
        message: "Are you sure you want to quit?",
        onConfirm: () =>
        {
            m_ui.PopUiElementFromCanvasIfMatch<ConfirmationDialog>(
                CanvasType.Front, blockingMode: InputBlockingMode.BlockNone);
            Application.Quit();
        },
        onCancel: () =>
        {
            m_ui.PopUiElementFromCanvasIfMatch<ConfirmationDialog>(
                CanvasType.Front, blockingMode: InputBlockingMode.BlockNone);
        }
    );

    m_ui.PushUiElementIntoCanvasCallback(
        dialog, CanvasType.Front,
        StackVisibilityMode.AllVisible,
        InputBlockingMode.BlockBelowTop,
        callback: () => dialog.Animate("Enter")
    );
}

```

Pattern 4: Tooltip on Hover

```

public class TooltipManager
{
    private readonly IStackableUiDirector m_ui;
    private TooltipElement m_current;

    public TooltipManager(IStackableUiDirector ui) => m_ui = ui;

    public void Show(string text)
    {
        if (m_current != null) return; // already showing

        m_current = new TooltipElement(text);
        m_ui.PushUiElementIntoCanvas(m_current, CanvasType.Over,
StackVisibilityMode.AllVisible);
    }

    public void Hide()
    {
        if (m_current == null) return;
        m_ui.RemoveUiElementFromCanvas(m_current, CanvasType.Over);
        m_current = null;
    }
}

```

Pattern 5: Loading Screen with Asset Streaming

```

public async void TransitionToGameplay()
{
    // 1. Push loading screen, blocking all input.
    var loading = new LoadingScreenElement();
    m_ui.PushUiElementIntoCanvasCallback(
        loading, CanvasType.Loading,
        StackVisibilityMode.AllVisible,
        InputBlockingMode.BlockAll,
        callback: async () =>
        {
            loading.Animate("Enter");

            // 2. Load the scene additively.
            await SceneManager.LoadSceneAsync("Gameplay", LoadSceneMode.Additive);

            // 3. Clear gameplay UI from previous session, if any.
            m_ui.ClearCanvas(CanvasType.Middle);

            // 4. Push the HUD.
            m_ui.PushUiElementIntoCanvas(new HudElement(), CanvasType.Middle);

            // 5. Remove loading screen and restore input.
            m_ui.PopUiElementFromCanvasIfMatch<LoadingScreenElement>(
                CanvasType.Loading, blockingMode: InputBlockingMode.BlockNone);
        }
    );
}

```

20. Pitfalls & Gotchas

PopUiElementCallback Does Not Fire If the Stack Is Empty After Pop

If the element you pop is the last one in the canvas, the current implementation returns early before invoking the callback. Always account for this edge case, or ensure the stack is never empty when you rely on the callback firing.

```
// Safe pattern: check before relying on the callback.
if (director.AnyElementInCanvas(CanvasType.Front))
{
    director.PopUiElementFromCanvasCallback(CanvasType.Front, () =>
    {
        // This fires only if the stack was non-empty.
    });
}
```

Consistency of `StackVisibilityMode` Across Push and Pop

The visibility mode is re-applied on every operation — not stored per-element. If you push with `TopOnly` and pop with `AllVisible`, the newly exposed top element will be set active. This is intentional, but it can produce unexpected visual results if you mix modes on the same canvas without thinking it through. Establish a convention per canvas layer and stick to it.

Duplicate Element Guard

The system checks if an element instance is already in the stack before pushing. If you try to push the same C# object reference twice, it is silently skipped with a warning. This means you must create a **new instance** for each push, even if two popups use the same prefab:

```
// WRONG: pushing the same instance twice.
var popup = new MyPopup();
director.PushUiElementIntoCanvas(popup, CanvasType.Front);
director.PushUiElementIntoCanvas(popup, CanvasType.Front); // ← skipped, warning
logged

// CORRECT: a new instance each time.
director.PushUiElementIntoCanvas(new MyPopup(), CanvasType.Front);
director.PushUiElementIntoCanvas(new MyPopup(), CanvasType.Front);
```

Async Push and Scene Unloading

If a scene is unloaded while an async push is in flight (e.g. `LoadAssetAsync` has not completed yet), the callback will fire on an already-destroyed canvas. Guard against this in your loader or in `OnPrefabLoaded` by checking whether the director is still initialized:

```
director.PushUiElementIntoCanvasCallback(element, CanvasType.Loading, callback: ()
=>
{
    if (!director.Initialized) return; // scene was unloaded mid-load
    // safe to proceed
});
```

OnPoppedFromStack and MonoBehaviour Coroutines

`SampleStackableElement.Animate` starts a coroutine via the `SampleStackableGuiDirector` singleton. If `OnPoppedFromStack` is called during scene teardown (e.g. from `OnDestroy`), the singleton may still be alive but the coroutine will fail because the hosting `MonoBehaviour` is being destroyed. The symptom is a Unity warning: *"Coroutine couldn't be started because the game object is being destroyed."*

The safest fix is to guard against this in your element's `OnPoppedFromStack`:

```
public override void OnPoppedFromStack()
{
    // Check that the coroutine host is alive before animating.
    if (SampleStackableGuiDirector.Active)
        Animate("Exit", Destroy);
    else
        Destroy(); // skip animation during teardown
}
```

ReleasePrefab Is Called Immediately on Pop

`ReleasePrefab` is called on the prefab reference the moment an element is removed from the stack — before `OnPoppedFromStack`'s exit animation plays out. With `Resources` this is harmless (the prefab is not unloaded until `Resources.UnloadUnusedAssets`). With `Addressables`, releasing the handle immediately can cause the asset to be unloaded while the exit animation is still running and referencing it. Defer the release by holding the handle until after the animation completes, or override `ReleasePrefab` in your loader to defer based on a frame delay.

End of documentation.